3. We would like to check whether the correlation between variable of interest and outcome

variable reflect the true relationship rather than the spurious correlation. One way is to change the DV to see whether the treatment also influence other DV. If the mechanism is true, we should observe null effect between the treament and different DVS. Another way to do the falsifiction test is to change a different treatment, and we should observed null effect.

One reason that may lead to the failure of a placebo test is the measurement error. The noise in the placebo outcome or treatment variable may create spurious correlations even when the true causal effect is zero.

In [39]:
```
# ----------------------------------------------------------------------
# Setup
# ----------------------------------------------------------------------
import os
import numpy as np
import pandas as pd
import matplotlib
from pandas.plotting import table

matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Reproducibility
np.random.seed(123)

from pathlib import Path

BASE = Path("/Users/a75700/Desktop/soda_501/soda_501/week 9 data quality"

DATA_RAW = BASE / "data_raw"
DATA_PROCESSED = BASE / "data_processed"
FIGURES = BASE / "figures"
OUTPUTS = BASE / "outputs"
SRC = BASE / "src"

for p in [DATA_RAW, DATA_PROCESSED, FIGURES, OUTPUTS, SRC]:
    p.mkdir(parents=True, exist_ok=True)
    print(f"OK: {p}")

# 4. 2. Using outputs/measurement error results.csv, make a clean table t
# (a) tau oracle mean, tau naive mean, tau cal mean, and
# (b) beta oracle mean, beta naive mean, beta cal mean
df = pd.read_csv('OUTPUTS/measurement_error_results.csv') # read csv file
variables = ["sigma_u","tau_oracle_mean", "tau_naive_mean", "tau_cal_mean
             "beta_oracle_mean", "beta_naive_mean", "beta_cal_mean"] # va
results = df[variables]
print("Table 1: Measurement Error Simulation Results")
print(results.to_string(index=False)) # present in table
```

```
OK: /Users/a75700/Desktop/soda_501/soda_501/week 9 data quality/data_raw
OK: /Users/a75700/Desktop/soda_501/soda_501/week 9 data quality/data_proce
ssed
OK: /Users/a75700/Desktop/soda_501/soda_501/week 9 data quality/figures
OK: /Users/a75700/Desktop/soda_501/soda_501/week 9 data quality/outputs
OK: /Users/a75700/Desktop/soda_501/soda_501/week 9 data quality/src
Table 1: Measurement Error Simulation Results
 sigma_u  tau_oracle_mean  tau_naive_mean  tau_cal_mean  beta_oracle_mean
beta_naive_mean  beta_cal_mean
     0.0         0.973679        0.973679      0.973679          1.015446
1.015446       1.015446
     0.2         0.973679        1.009563      1.009563          1.015446
0.968523       1.010455
     0.5         0.973679        1.175672      1.175672          1.015446
0.774379       0.977539
     1.0         0.973679        1.445083      1.445083          1.015446
0.449526       0.918711
     2.0         0.973679        1.679230      1.679230          1.015446
0.170691       0.875426
```

3. In the simulation, σ_u is the standard deviation of the error and τ is the estimated treated effect.

Figure one reflects that as σ_u increase, the τ also changes. The oracle is stable at the true value regardless of the value of σ_u, because it uses the true value to estimate. However, the naive estimates deviate from the true estimate as σ_u increase because it it uses the noisy x_obs. The residual confounding makes the treatment coefficient biased. The placebo outcome estimate stays near zero across all σ_u levels, which is reassuring, it confirms that the bias in the naive model is specifically due to residual confounding, not some mechanical artifact of measurement error itself.

Figure (b) shows the attenuation bias in the confounder coefficient. As σ_u increases, the naive estimate of beta shrinks toward zero, while the oracle estimate stays near the true value of 1.0. This happens because when x_obs is a noisy version of x_true, the regression is influenced by more noise in the confounder, causing the estimated coefficient to be pulled toward zero. The calibration estimator partially corrects this by first regressing x_true on x_obs in a validation sample to recover a cleaner prediction x_hat, bringing both tau and beta estimates closer to the oracle.

In [46]:

```
# Helper: OLS via numpy (no statsmodels needed)
# Returns dict with coefficient names and values

# We use numpy.linalg.lstsq to fit OLS: y = X @ beta
# This avoids requiring statsmodels or scipy.

# Part 1: A simple data generating process with confounding

# We simulate:
#   X_true: a true confounder (unobserved truth)
#   D: a treatment/exposure correlated with X_true
#   Y: an outcome affected by both D and X_true
#
# Then we observe:
```

```
#   X_obs = X_true + U   (measurement error in X)

n = 5000

# True confounder
x_true = np.random.normal(loc=0.0, scale=1.0, size=n)

# Treatment assignment correlated with x_true (logistic link)
# (This creates confounding: D is not independent of x_true.)
logit_p = 1.0 * x_true
p = 1.0 / (1.0 + np.exp(-logit_p))
d = np.random.binomial(n=1, p=p, size=n)

# True outcome model
tau = 1.0     # true effect of D on Y
beta = 1.0    # effect of X_true on Y
eps_y = np.random.normal(loc=0.0, scale=1.0, size=n)

y = tau * d + beta * x_true + eps_y

# Placebo outcome (negative control outcome): NOT affected by D by constr
eps_pl = np.random.normal(loc=0.0, scale=1.0, size=n)
y_placebo = 0.0 * d + beta * x_true + eps_pl

df_base = pd.DataFrame(
    {
        "y": y,
        "y_placebo": y_placebo,
        "d": d,
        "x_true": x_true,
    }
)

print("\n--- Data preview ---")
print(df_base.head())
print("\nTreatment rate:", round(df_base["d"].mean(), 4))

# Part 2: Validation set
# For each sigma_u, we will estimate:
#   (A) Oracle model: y ~ d + x_true           (best case; uses truth)
#   (B) Naive model:  y ~ d + x_obs            (what you do with a noisy
#   (C) Calibration:  estimate x_true ~ x_obs in a validation sample, pre

sigma_u = 1 # set sigma_u =1
R = 200  # repetitions

# validtaion list
validation_share_list = [0.05,0.2,0.5]

# Storage
rows = []

print("\n--- Running validation sample simulations ---")
for validation_share in validation_share_list:
    tau_oracle_list = []
    tau_naive_list = []
    tau_cal_list = []
    tau_placebo_list = []

    beta_oracle_list = []
```

```
    beta_naive_list = []
    beta_cal_list = []

    for r in range(R):
        # random error
        u = np.random.normal(loc=0.0, scale=sigma_u, size=n)
        x_obs = x_true + u # x_obs
        validation_idx = np.random.choice(np.arange(n), size=int(validati
        is_validation = np.zeros(n, dtype=bool)
        is_validation[validation_idx] = True
        # Build design matrices with intercept column
        ones = np.ones(n)

        # (A) Oracle regression: y ~ 1 + d + x_true
        X_oracle = np.column_stack([ones, d, x_true])
        coef_oracle, _, _, _ = np.linalg.lstsq(X_oracle, y, rcond=None)
        # coef_oracle: [intercept, d, x_true]
        tau_oracle_list.append(coef_oracle[1])
        beta_oracle_list.append(coef_oracle[2])

        # (B) Naive regression: y ~ 1 + d + x_obs
        X_naive = np.column_stack([ones, d, x_obs])
        coef_naive, _, _, _ = np.linalg.lstsq(X_naive, y, rcond=None)
        tau_naive_list.append(coef_naive[1])
        beta_naive_list.append(coef_naive[2])

        # (C) Regression calibration (validation subsample):
        #     estimate x_true ~ x_obs on validation sample, predict x_hat
        ones_val = np.ones(int(validation_share * n))
        X_cal_val = np.column_stack([ones_val, x_obs[is_validation]])
        coef_cal, _, _, _ = np.linalg.lstsq(X_cal_val, x_true[is_validati
        x_hat = coef_cal[0] + coef_cal[1] * x_obs

        X_calibrated = np.column_stack([ones, d, x_hat])
        coef_calibrated, _, _, _ = np.linalg.lstsq(X_calibrated, y, rcond
        tau_cal_list.append(coef_calibrated[1])
        beta_cal_list.append(coef_calibrated[2])

        # Outcome placebo: y_placebo ~ 1 + d + x_obs
        X_placebo = np.column_stack([ones, d, x_obs])
        coef_placebo, _, _, _ = np.linalg.lstsq(X_placebo, y_placebo, rco
        tau_placebo_list.append(coef_placebo[1])

    # Summaries per sigma_u
    rows.append(
        {
            "validation_share": validation_share,
            "tau_true": tau,
            "tau_oracle_mean": float(np.mean(tau_oracle_list)),
            "tau_naive_mean": float(np.mean(tau_naive_list)),
            "tau_cal_mean": float(np.mean(tau_cal_list)),
            "tau_placebo_mean": float(np.mean(tau_placebo_list)),
            "tau_oracle_q025": float(np.quantile(tau_oracle_list, 0.025))
            "tau_oracle_q975": float(np.quantile(tau_oracle_list, 0.975))
            "tau_naive_q025": float(np.quantile(tau_naive_list, 0.025)),
            "tau_naive_q975": float(np.quantile(tau_naive_list, 0.975)),
            "tau_cal_q025": float(np.quantile(tau_cal_list, 0.025)),
            "tau_cal_q975": float(np.quantile(tau_cal_list, 0.975)),
            "beta_true": beta,
            "beta_oracle_mean": float(np.mean(beta_oracle_list)),
```

```
            "beta_naive_mean": float(np.mean(beta_naive_list)),
            "beta_cal_mean": float(np.mean(beta_cal_list)),
        }
    )

    print(f"  done sigma_u={sigma_u}")

results = pd.DataFrame(rows)
print("\n--- Summary (means) ---")
print(results[["validation_share", "tau_true", "tau_oracle_mean", "tau_na

results.to_csv(OUTPUTS/"validation_share_results.csv", index=False)

#######################################################################
```

```
--- Data preview ---
          y  y_placebo  d    x_true
0  2.921217   0.122286  1  0.360747
1  1.717122   0.875510  1  0.803191
2  2.157886  -0.788934  1  0.869571
3 -1.501872  -2.013753  0 -0.959027
4 -0.628774  -1.196969  0 -1.197800

Treatment rate: 0.4946

--- Running validation sample simulations ---
  done sigma_u=1
  done sigma_u=1
  done sigma_u=1

--- Summary (means) ---
 validation_share  tau_true  tau_oracle_mean  tau_naive_mean  tau_cal_mean
tau_placebo_mean
             0.05       1.0         1.039989        1.465599      1.465599
0.460297
             0.20       1.0         1.039989        1.465616      1.465616
0.461278
             0.50       1.0         1.039989        1.463269      1.463269
0.459320
```

Across all three validation share settings (0.05, 0.20, 0.50), the calibrated treatment estimate tau_cal_mean remains nearly identical to the naive estimate tau_naive_mean. Both substantially above the true value of 1.0, while the oracle estimate stays close to 1.0 throughout.

In principle, calibration can help when the confounder is measured with error by using a validation subsample to learn the mapping from noisy to true confounder values, producing x_hat that better approximates x_true. However, It is not magic: in this simulation, because the measurement error is classical and additive (x_obs = x_true + u), the calibration slope shrinks x_obs toward zero, but this shrinkage is exactly offset when the outcome regression re-estimates the confounder coefficient, leaving tau_cal unchanged relative to tau_naive. Calibration also relies critically on the assumption that the validation sample is representative of the full sample and that the error structure is identical across both.

In [48]:
```
# ---------------------------------------------------------------------------
# Part 4: Treatment permutation placebo (randomization inference)
# ---------------------------------------------------------------------------


sigma_u_perm = 1.0
u_perm = np.random.normal(loc=0.0, scale=sigma_u_perm, size=n)
x_obs_perm = x_true + u_perm

ones = np.ones(n)
X_obs = np.column_stack([ones, d, x_obs_perm])
coef_obs, _, _, _ = np.linalg.lstsq(X_obs, y, rcond=None)
tau_hat_obs = float(coef_obs[1])

print("\n--- Permutation placebo setup ---")
print("sigma_u used:", sigma_u_perm)
print("Observed tau_hat (naive model):", round(tau_hat_obs, 4))

B = 500
tau_perm = []

for b in range(B):
    d_perm = np.random.permutation(d)
    X_b = np.column_stack([ones, d_perm, x_obs_perm])
    coef_b, _, _, _ = np.linalg.lstsq(X_b, y, rcond=None)
    tau_perm.append(float(coef_b[1]))

tau_perm = np.array(tau_perm)

# Empirical two-sided p-value
p_emp = (1.0 + np.sum(np.abs(tau_perm) >= np.abs(tau_hat_obs))) / (B + 1.
print("Empirical p-value (two-sided):", round(p_emp, 4))

perm_df = pd.DataFrame({"tau_perm": tau_perm})
perm_df.to_csv("outputs/permutation_tau_distribution.csv", index=False)

# Plot permutation distribution + observed line
plt.figure(figsize=(8, 5))
plt.hist(tau_perm, bins=30, alpha=0.8)
plt.axvline(tau_hat_obs, linestyle="--", linewidth=2, label=f"Observed ta
plt.axvline(-tau_hat_obs, linestyle="--", linewidth=1)
plt.title(f"Treatment permutation placebo (sigma_u={sigma_u_perm})\nEmpir
plt.xlabel("Coefficient on permuted treatment")
plt.ylabel("Count")
plt.legend()
plt.tight_layout()
plt.savefig("figures/permutation_placebo_tau_hist.png", dpi=200)
plt.close()
```

```
--- Permutation placebo setup ---
sigma_u used: 1.0
Observed tau_hat (naive model): 1.4383
Empirical p-value (two-sided): 0.002
```

The permutation test builds a null distribution by randomly shuffling the treatment variable d 500 times and re-estimating τ each time, representing what estimates would look like if there were no causal effect of D on Y. (a) The null hypothesis is that treatment assignment is independent of the outcome Y. In other words, D has no

causal effect on Y after controlling for x_obs. (b) The observed naive estimate tau_hat_obs falls far to the right of the permutation distribution, and the empirical p-value is very small (close to 0), meaning it would be extremely unlikely to observe an estimate this large by chance alone if the null were true. (c) Placebo tests can detect pipeline problems: for example, if a coding error caused y to be constructed using d twice, or if there is data leakage where future outcomes contaminate the treatment variable, the outcome placebo tau_placebo would be significantly different from zero even though it was designed to have no treatment effect — flagging that something in the data pipeline is wrong.